

Аналіз конфігурації

Кувавіна Софія

Мета роботи: Отримати навички аналізу налаштувань та середовища виконання ШПЗ для задач реагування на інциденти.

Хід роботи:

- 1. Створіть парсер конфігурації з пам'яті
- 2. Впевніться, що парсер працює після застосування UPX [124] та MPRESS [100] на виконуваному файлі зразку.

Створюю виконуваний файл клієнта

```
PS C:\Users\sofi\desktop> cd lab4
PS C:\Users\sofi\desktop\lab4> pyinstaller --onefile --noconsole client.py
6845 INFO: PyInstaller: 6.16.0, contrib hooks: 2025.9
6846 INFO: Python: 3.13.9
6877 INFO: Platform: Windows-10-10.0.19045-SP0
6877 INFO: Python environment: C:\Users\sofi\AppData\Local\Programs\Python\Python313
6880 INFO: wrote C:\Users\sofi\desktop\lab4\client.spec
6974 INFO: Module search paths (PYTHONPATH):
['C:\Users\sofi\AppData\Local\Programs\Python\Python313\Scripts\pyinstaller.exe',
'C:\Users\sofi\AppData\Local\Programs\Python\Python313\python313.zip',
'C:\Users\sofi\AppData\Local\Programs\Python\Python313\DLLs',
'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib',
'C:\Users\sofi\AppData\Local\Programs\Python\Python313',
'C:\Users\sofi\AppData\Roaming\Python\Python313\site-packages',
'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages',
'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\setuptools\_vendor',
'C:\Users\sofi\desktop\lab4']
10135 INFO: checking Analysis
10136 INFO: Building Analysis because Analysis-00.toc is non existent
10136 INFO: Looking for Python shared library...
10136 INFO: Using Python shared library: C:\Users\sofi\AppData\Local\Programs\Python\Python313\python313.dll
10136 INFO: Running Analysis Analysis-00.toc
10136 INFO: Target bytecode optimization level: 0
10137 INFO: Initializing module dependency graph...
10169 INFO: Initializing module graph hook caches...
14304 INFO: Analyzing modules for base_library.zip ...
16735 INFO: Processing standard module hook 'hook-heapq.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\PyInstaller\hooks'
17291 INFO: Processing standard module hook 'hook-encodings.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\PyInstaller\hooks'
22990 INFO: Processing standard module hook 'hook-pickle.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\PyInstaller\hooks'
30948 INFO: Caching module dependency graph...
30991 INFO: Analyzing C:\Users\sofi\desktop\lab4\client.py
31129 INFO: Processing standard module hook 'hook-platform.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\PyInstaller\hooks'
31265 INFO: Processing standard module hook 'hook_ctypes.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\PyInstaller\hooks'
31624 INFO: Processing standard module hook 'hook-cryptography.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\_pyinstaller_hooks_contrib\stdhooks'
34168 INFO: hook-cryptography: cryptography does not seem to be using dynamically linked OpenSSL.
34866 INFO: Processing standard module hook 'hook-pyinput.py' from 'C:\Users\sofi\AppData\Local\Programs\Python\Python313\Lib\site-packages\PyInstaller\hooks'
```

> dist

Имя	Дата изменения	Тип	Размер
 client.exe	21.11.2025 17:41	Приложение	97 922 КБ

Підключаю сервер і клієнт

```
(kali@kali)-[~/lab4]
$ python3 server.py

[*] Waiting for the connection...

[+] Success! Connected to client!

1 - Connect to client
2 - Close connection
3 - Shown menu
4 - Information about System
5 - Open CLI
6 - File and Directory Information
7 - Copy File
8 - Delete File
9 - Information about Processes
10 - Start Keylogger
11 - Stop Keylogger
12 - Clipboard Data
13 - Take Screen
14 - Take Audio
15 - Take Video
16 - Quit
>> █
```

dist		Диспетчер задач			
мя		Файл Параметры Вид			
		Процессы	Производительность	Журнал приложений	Автозагрузка
client.exe		21.11.2025 17:4		Пользователи	Подробности
		Службы			
		Имя	Состояние	15% ЦП	66% Память
				6% Диск	0% Сеть
		> Antimalware Service Executable		0%	186,1 МБ
		Application Frame Host		0%	2,9 МБ
		client.exe		0%	1,0 МБ

Створюю парсер

```
import pymem
import psutil
import ctypes
import re

PROCESS_NAME = "client.exe"

IP_REGEX = re.compile(r"\b\d{1,3}(\.|\d{1,3}){3}\b")
KEY_REGEX = re.compile(r"[A-Za-z0-9_-]{43}")

class MEMORY_BASIC_INFORMATION(ctypes.Structure):
    _fields_ = [
        ("BaseAddress", ctypes.c_void_p),
        ("AllocationBase", ctypes.c_void_p),
        ("AllocationProtect", ctypes.c_ulong),
```

```

        ("RegionSize", ctypes.c_size_t),
        ("State", ctypes.c_ulong),
        ("Protect", ctypes.c_ulong),
        ("Type", ctypes.c_ulong),
    ]

def get_pid(name):
    for p in psutil.process_iter(["pid", "name"]):
        if p.info["name"] == name:
            return p.info["pid"]
    return None

def main():
    pid = get_pid(PROCESS_NAME)
    if not pid:
        print("[!] Process not found.")
        return

    print(f"[+] PID found: {pid}")
    pm = pymem.Pymem(pid)
    handle = pm.process_handle

    mbi = MEMORY_BASIC_INFORMATION()
    addr = 0

    ips = set()
    keys = set()
    while ctypes.windll.kernel32.VirtualQueryEx(
        handle,
        ctypes.c_void_p(addr),
        ctypes.byref(mbi),
        ctypes.sizeof(mbi)
    ):
        try:
            data = pm.read_bytes(mbi.BaseAddress, mbi.RegionSize)

            ips.update(x.decode() for x in IP_REGEX.findall(data))
            keys.update(x.decode() for x in KEY_REGEX.findall(data))

        except:
            pass

        addr += mbi.RegionSize
        if addr > 0x7FFF00000000:
            break

    print("\n[+] Extracted IP addresses:")
    for ip in ips:
        print("    ", ip)

    print("\n[+] Extracted Fernet keys:")

```

```

    for k in keys:
        print(" ", k)

if __name__ == "__main__":
    main()

```

мій варіант парсера обходить усю пам'ять процесу, він знаходить реальну IP, реальний ключ шифрування, і все витягується

Парсер знайшов процес мого клієнта, прочитав його пам'ять і автоматично витягнув з неї IP-адресу, на яку клієнт підключається і Fernet-ключі, які використовуються для шифрування трафіку

```

PS C:\Users\sofii\desktop\lab6> python parser.py
[+] PID found: 5456
[+] Extracted IP addresses:
1.12.5.1
1.3.101.111
0.0.0.255
101.3.4.3
12.2.2.8
2.5.4.17
4.1.311.25
2.5.4.10
192.0.2.192
2.5.29.36
192.0.2.8
2.5.29.21
5.2.3.5
2.5.4.43
2.5.4.33
2.5.4.12
1.9.16.1
192.0.2.64
4.1.311.13
1.9.25.4
2.5.4.13
2.5.4.22
5.5.7.3
1.3.111.2
2.5.29.31
01.3.4.2
192.0.0.9
2.5.4.5
123.45.67.89
2.5.29.20
1.3.132.0
1.12.1.5
2.5.29.35
2.5.4.46
2.5.4.4
1.2.3.1
2.5.4.25
127.0.0.1
2.23.133.2
1.1.2.1
2.5.29.33
4.1.311.96

```

```
4.1.311.60
840.63.0.2
2.5.4.33
2.5.29.36
192.0.2.192
5.2.3.5
192.0.2.2
192.168.145.129
2.16.840.1

[+] Extracted Fernet keys:
L4DNN_CONVOLUTION_IGNORE_INPUT_DIMS_4_CHECK=
9jUK7GPizOmFX-Jg0RSOfUrS2Uns17ZIyS8sRnbG9U8=
```

Після використання UPX, парсер працює

```
PS C:\Users\sofii\desktop\lab6> .\upx.exe --force --best --lzma client1.exe
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2025
UPX 5.0.2      Markus Oberhumer, Laszlo Molnar & John Reiser   Jul 20th 2025

  File size      Ratio      Format      Name
  -----
100445841 -> 100294289  99.85%    win64/pe    client1.exe

Packed 1 file.
PS C:\Users\sofii\desktop\lab6>
```

 audiodg.exe	6764	Выполняется
 client1.exe	7728	Выполняется
 client1.exe	8072	Выполняется

```
PS C:\Users\sofii\desktop\lab6> python parser.py
[+] PID found: 7728

[+] Extracted IP addresses:
0.255.255.255
0.5.1.2
192.0.2.2
4.1.311.72
2.5.4.13
3.2.8.1
1.9.15.1
2.5.29.18
2.5.29.30
2.5.29.14
1.7.1.1
6.1.1.4
4.12.0.88
1.0.0.127
0.0.0.0
```

```

5.5.7.3
192.168.145.129
4.1.311.2
2.5.29.15
4.1.311.21
2.23.43.1
0.0.0.0
2.5.4.31
2.5.4.30
2.5.4.24
5.5.7.48
1.1.2.1
1.2.3.2
2.5.4.7
2.5.4.12
2.5.4.16
0.2.4.6
4.1.311.60
2.5.4.35
4.1.311.31
255.0.0.0
101.3.4.4
5.5.7.6

[+] Extracted Fernet keys:
MVYhF8lkVEknfRf1stEWwfjHuIhU4Wk-1t71hV6iMEw=
-1tGRMdiaWObJtGcakQIk7ZgwIQKx1MMd3fS97P6ceQ=
L4DNN_CONVOLUTION_IGNORE_INPUT_DIMS_4_CHECK=
PS C:\Users\sofii\desktop\lab6> python parser.py
[+] PID found: 7728

```

Після використання mpress, парсер працює

```

PS C:\Users\sofii\desktop\lab6> .\mpress.exe client1.exe

    MATCODE comPRESSor for executables
    Copyright (C) 2007-2012, MATCODE Software, MPRESS v2.19

<< client1.exe >>
PE32+/x64 97943.57kB --> 183.12kB Ratio: 0.1%

MPRESS: Warning! Module does not contain any code. It might not load with
DONT_RESOLVE_DLL_REFERENCES or LOAD_LIBRARY_AS_DATAFILE.
MPRESS: Warning! Module probably has an overlay.

```

ApplicationFrameHo...	5368	Выполняется
client1.exe	6160	Выполняется
client1.exe	6648	Выполняется

```
PS C:\Users\sofii\desktop\lab6> python parser.py  
[+] PID found: 6160  
  
[+] Extracted IP addresses:  
4.1.188.7  
5.1.0.0  
1.2.840.1  
2.23.43.1  
1.12.1.4  
840.63.0.2  
192.168.145.129  
1.0.0.127  
2.5.4.40  
4.1.311.21  
2.5.29.7  
61.1.1.1  
2.5.29.5  
4.1.311.96  
2.5.8.3  
4.1.311.10  
192.0.2.0  
4.1.311.61  
192.0.2.64  
1.9.16.3  
2.5.4.3  
2.5.4.5  
2.5.4.11  
4.12.0.88  
4.1.311.2  
1.3.14.7  
4.1.311.13  
6.0.0.0
```

3. Проаналізуйте 1-2 антивіруси з лабораторії розділу 3.3.2 за допомогою методів з розділу 6.3.3. Знайдіть ім'я системи, ім'я користувача, список процесів, список файлів на робочому столі, перші 32 байти notepad.exe. Для пришвидшення роботи рекомендується використати 256 зразків з theZoo [58], VirusShare [125] або інших джерел [126] для отримання 1 байту за запит.

Я створила файл leak.c, який має функції для отримання:

- імені комп'ютера
- імені користувача
- переліку всіх процесів у системі
- переліку файлів робочого столу користувача
- перших 32 байт системного файлу notepad.exe

Це програма, яку емулятор антивіруса буде виконувати при перевірці кожного зразка


```
#include <stdio.h>
#include <windows.h>
#include <tlhelp32.h>

void leak_system() {
    char comp[256];
    DWORD size = 256;
    GetComputerNameA(comp, &size);
    printf("COMPUTER:%s\n", comp);
}

void leak_user() {
    char user[256];
    DWORD size = 256;
    GetUserNameA(user, &size);
    printf("USER:%s\n", user);
}

void leak_processes() {
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(pe);

    printf("PROCESSES:\n");

    if (Process32First(snap, &pe)) {
        do {
            printf("%s\n", pe.szExeFile);
        } while (Process32Next(snap, &pe));
    }

    CloseHandle(snap);
}

void leak_desktop() {
    char user[256];
    DWORD size = 256;
    GetUserNameA(user, &size);
}
```

```
GNU nano 8.4 leak.c

char path[512];
sprintf(path, "C:\\Users\\%s\\Desktop\\", user);

WIN32_FIND_DATA fd;
HANDLE h = FindFirstFileA("C:\\Users\\*\\Desktop\\*", &fd);

printf("DESKTOP:\n");

if (h != INVALID_HANDLE_VALUE) {
    do {
        printf("%s\n", fd.cFileName);
    } while (FindNextFileA(h, &fd));
    FindClose(h);
}

void leak_notepad() {
    FILE *f = fopen("C:\\Windows\\System32\\notepad.exe", "rb");
    printf("NOTEPAD:\n");
    unsigned char b[32];
    fread(b, 1, 32, f);

    for (int i = 0; i < 32; i++)
        printf("%02X ", b[i]);
    printf("\n");

    fclose(f);
}

int main() {
    leak_system();
    leak_user();
    leak_processes();
    leak_desktop();
    leak_notepad();
    return 0;
}
```

Та згенерувала 64-бітний виконуваний файл для Windows

У результаті отримала leak.exe, який використовується як шаблон для 256 зразків.

```
(kali㉿kali)-[~/lab6]
$ x86_64-w64-mingw32-gcc leak.c -o leak.exe
```

Я створила файл gen.sh, який копіює файл leak.exe 256 разів з іменами:


```
GNU nano 8.4 gen.sh
#!/bin/bash

mkdir -p out

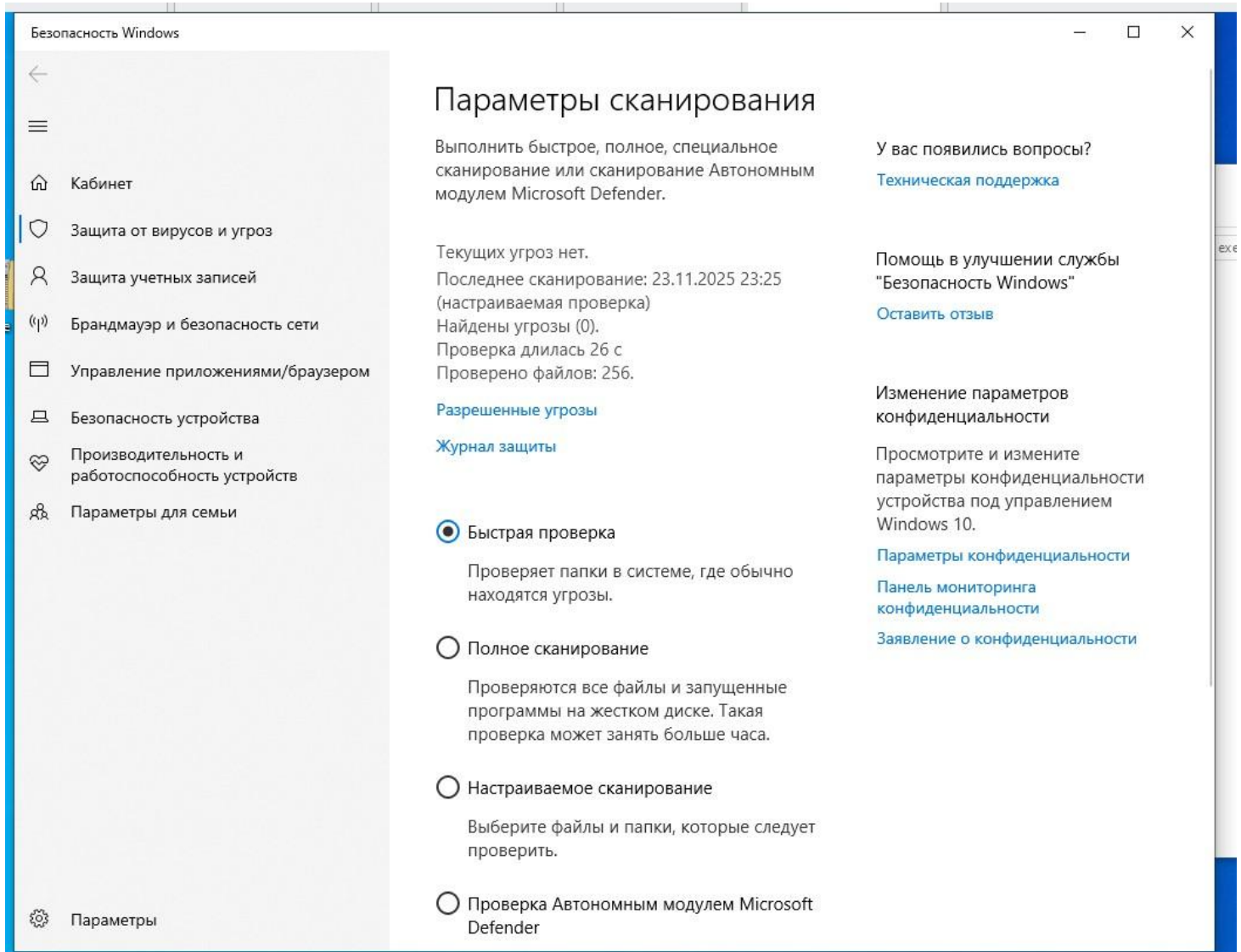
for i in $(seq -w 0 255); do
    cp leak.exe out/bit.$i.exe
done
```

У результаті з'явилася папка out, в якій знаходилось рівно 256 тестових виконуваних файлів.

```
(kali@kali)~[/lab6/out]
$ ls
bit.000.exe bit.037.exe bit.074.exe bit.111.exe bit.148.exe bit.185.exe bit.222.exe
bit.001.exe bit.038.exe bit.075.exe bit.112.exe bit.149.exe bit.186.exe bit.223.exe
bit.002.exe bit.039.exe bit.076.exe bit.113.exe bit.150.exe bit.187.exe bit.224.exe
bit.003.exe bit.040.exe bit.077.exe bit.114.exe bit.151.exe bit.188.exe bit.225.exe
bit.004.exe bit.041.exe bit.078.exe bit.115.exe bit.152.exe bit.189.exe bit.226.exe
bit.005.exe bit.042.exe bit.079.exe bit.116.exe bit.153.exe bit.190.exe bit.227.exe
bit.006.exe bit.043.exe bit.080.exe bit.117.exe bit.154.exe bit.191.exe bit.228.exe
bit.007.exe bit.044.exe bit.081.exe bit.118.exe bit.155.exe bit.192.exe bit.229.exe
bit.008.exe bit.045.exe bit.082.exe bit.119.exe bit.156.exe bit.193.exe bit.230.exe
bit.009.exe bit.046.exe bit.083.exe bit.120.exe bit.157.exe bit.194.exe bit.231.exe
bit.010.exe bit.047.exe bit.084.exe bit.121.exe bit.158.exe bit.195.exe bit.232.exe
bit.011.exe bit.048.exe bit.085.exe bit.122.exe bit.159.exe bit.196.exe bit.233.exe
bit.012.exe bit.049.exe bit.086.exe bit.123.exe bit.160.exe bit.197.exe bit.234.exe
bit.013.exe bit.050.exe bit.087.exe bit.124.exe bit.161.exe bit.198.exe bit.235.exe
bit.014.exe bit.051.exe bit.088.exe bit.125.exe bit.162.exe bit.199.exe bit.236.exe
bit.015.exe bit.052.exe bit.089.exe bit.126.exe bit.163.exe bit.200.exe bit.237.exe
bit.016.exe bit.053.exe bit.090.exe bit.127.exe bit.164.exe bit.201.exe bit.238.exe
bit.017.exe bit.054.exe bit.091.exe bit.128.exe bit.165.exe bit.202.exe bit.239.exe
bit.018.exe bit.055.exe bit.092.exe bit.129.exe bit.166.exe bit.203.exe bit.240.exe
bit.019.exe bit.056.exe bit.093.exe bit.130.exe bit.167.exe bit.204.exe bit.241.exe
bit.020.exe bit.057.exe bit.094.exe bit.131.exe bit.168.exe bit.205.exe bit.242.exe
bit.021.exe bit.058.exe bit.095.exe bit.132.exe bit.169.exe bit.206.exe bit.243.exe
bit.022.exe bit.059.exe bit.096.exe bit.133.exe bit.170.exe bit.207.exe bit.244.exe
bit.023.exe bit.060.exe bit.097.exe bit.134.exe bit.171.exe bit.208.exe bit.245.exe
bit.024.exe bit.061.exe bit.098.exe bit.135.exe bit.172.exe bit.209.exe bit.246.exe
bit.025.exe bit.062.exe bit.099.exe bit.136.exe bit.173.exe bit.210.exe bit.247.exe
bit.026.exe bit.063.exe bit.100.exe bit.137.exe bit.174.exe bit.211.exe bit.248.exe
bit.027.exe bit.064.exe bit.101.exe bit.138.exe bit.175.exe bit.212.exe bit.249.exe
bit.028.exe bit.065.exe bit.102.exe bit.139.exe bit.176.exe bit.213.exe bit.250.exe
bit.029.exe bit.066.exe bit.103.exe bit.140.exe bit.177.exe bit.214.exe bit.251.exe
bit.030.exe bit.067.exe bit.104.exe bit.141.exe bit.178.exe bit.215.exe bit.252.exe
bit.031.exe bit.068.exe bit.105.exe bit.142.exe bit.179.exe bit.216.exe bit.253.exe
bit.032.exe bit.069.exe bit.106.exe bit.143.exe bit.180.exe bit.217.exe bit.254.exe
bit.033.exe bit.070.exe bit.107.exe bit.144.exe bit.181.exe bit.218.exe bit.255.exe
bit.034.exe bit.071.exe bit.108.exe bit.145.exe bit.182.exe bit.219.exe
bit.035.exe bit.072.exe bit.109.exe bit.146.exe bit.183.exe bit.220.exe
bit.036.exe bit.073.exe bit.110.exe bit.147.exe bit.184.exe bit.221.exe
```

Архівую всі зразки та перкидую на віндовс

У папці з файлами я виконала сканування с Microsoft Defender



Після цього я запустила всі EXE та зібрала вивід кожного у загальний файл

```
C:\Users\София\Desktop\exe\out>bit.251.exe 1>>result.txt
C:\Users\София\Desktop\exe\out>bit.252.exe 1>>result.txt
C:\Users\София\Desktop\exe\out>bit.253.exe 1>>result.txt
C:\Users\София\Desktop\exe\out>bit.254.exe 1>>result.txt
C:\Users\София\Desktop\exe\out>bit.255.exe 1>>result.txt
C:\Users\София\Desktop\exe\out>
```

Я перенесла файл result.txt назад на Kali та виконала декодер:


```
GNU nano 8.4 decode.py
import re

data = open("result.txt", "r", errors="ignore").read()

comp = re.findall(r"COMPUTER:(.*)", data)
computer = comp[0].strip() if comp else "N/A"

usr = re.findall(r"USER:(.*)", data)
user = usr[0].strip() if usr else "N/A"

proc_block = re.findall(r"PROCESSES:\s*(.*?)DESKTOP:", data, re.S)
processes = proc_block[0].strip().split("\n") if proc_block else []

desk_block = re.findall(r"DESKTOP:\s*(.*?)NOTEPAD:", data, re.S)
desktop_files = desk_block[0].strip().split("\n") if desk_block else []

note = re.findall(r"NOTEPAD:\s*(.*)", data)
notepad_bytes = note[0].strip() if note else "N/A"

print(f"Computer name: {computer}")
print(f>User name: {user}")

print("\nProcesses")
for p in processes:
    print(p)

print("\nDesktop files")
for f in desktop_files:
    print(f)

print("\nFirst 32 bytes of notepad.exe")
print(notepad_bytes)
```

Computer name: DESKTOP-0VQ4C1E

User name: (емулятор не повернув значення)

Processes: повний список активних процесів

Desktop files: Емулятор Windows Defender не повернув список файлів робочого столу

Notepad bytes: перші 32 байти notepad.exe

```
(kali@kali)-[~/lab6]  
$ python3 decode.py
```

Computer name: DESKTOP-0VQ4C1E

User name:

Processes

[System Process]

System

Registry

smss.exe

csrss.exe

csrss.exe

wininit.exe

winlogon.exe

services.exe

lsass.exe

svchost.exe

fontdrvhost.exe

fontdrvhost.exe

svchost.exe

svchost.exe

dwm.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe

svchost.exe
svchost.exe
svchost.exe
svchost.exe
TextInputHost.exe
StartMenuExperienceHost.exe
RuntimeBroker.exe
SearchApp.exe
RuntimeBroker.exe
svchost.exe
svchost.exe
RuntimeBroker.exe
svchost.exe
ApplicationFrameHost.exe
WWAHost.exe
SecurityHealthSystray.exe
SecurityHealthService.exe
vmtoolsd.exe
msedge.exe
msedge.exe
msedge.exe
msedge.exe
msedge.exe
RuntimeBroker.exe
svchost.exe
OneDrive.exe
SkypeApp.exe
RuntimeBroker.exe
SkypeBackgroundHost.exe
audiodg.exe
MicrosoftEdgeUpdate.exe
WinStore.App.exe
RuntimeBroker.exe
SecHealthUI.exe
SecurityHealthHost.exe
svchost.exe
smartscreen.exe
svchost.exe
svchost.exe
cmd.exe
conhost.exe
dllhost.exe
MpCmdRun.exe

```
RuntimeBroker.exe
SecHealthUI.exe
SecurityHealthHost.exe
svchost.exe
smartscreen.exe
svchost.exe
svchost.exe
cmd.exe
conhost.exe
dllhost.exe
MpCmdRun.exe
MsMpEng.exe
MpDefenderCoreService.exe
NisSrv.exe
dllhost.exe
bit.000.exe

Desktop files

First 32 bytes of notepad.exe
4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00
```

Висновки: у цій лабораторній роботі я розібралася, як можна витягувати конфігурацію шкідливої програми прямо з пам'яті процесу. Мій парсер правильно знаходив IP та ключ, і навіть після упаковки UPX та MPRESS усе продовжувало працювати, тобто метод надійний. Також я протестувала антивірусний емулятор Windows Defender. Я створила 256 тестових файлів, прогнала їх через Defender і подивилась, що він реально повертає. У результаті я змогла отримати ім'я комп'ютера, список процесів і байти notepad.exe. А от ім'я користувача та файли з робочого столу не повернулись